

Claims_ejercicio

June 15, 2026

1 Análisis Exploratorio de Datos (EDA) - Detección de Fraude en Seguros de Auto

Este dataset contiene información sobre pólizas de seguro, perfiles de clientes, características de vehículos y detalles de siniestros automovilísticos. El objetivo principal es identificar patrones que diferencien los reclamos legítimos de los fraudulentos (`fraud_reported`).

1.1 Contenido

1. Clasificación y Explicación de las Columnas
2. Análisis Inicial
3. Relación con el Fraude
4. Pasos que faltan para profundizar

```
[ ]: # Cargamos librerías necesarias
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import datetime
from scipy import stats
from scipy.stats import mannwhitneyu, norm
from scipy.stats import chi2_contingency
from scipy.stats import pearsonr
from scipy.stats import spearmanr
from scipy.stats import kendalltau
from scipy.stats import anderson
import plotly.graph_objects as go
import plotly.express as px

from matplotlib.animation import FuncAnimation
rng = np.random.default_rng()
x = stats.uniform.rvs(size=75, random_state=rng)
sns.set(style="darkgrid")
```

```
[2]: # Cargamos la base de datos
df = pd.read_csv('insurance_claims.csv')
```

```
[3]: # Realizamos un primer vistazo a la base de datos
df.head()
```

```
[3]:
```

	months_as_customer	age	policy_number	policy_bind_date	policy_state	\
0	328	48	521585	17/10/2014	OH	
1	228	42	342868	27/06/2006	IN	
2	134	29	687698	06/09/2000	OH	
3	256	41	227811	25/05/1990	IL	
4	228	44	367455	06/06/2014	IL	

	policy_csl	policy_deductable	policy_annual_premium	umbrella_limit	\
0	250/500	1000	1406.91	0	
1	250/500	2000	1197.22	5000000	
2	100/300	2000	1413.14	5000000	
3	250/500	2000	1415.74	6000000	
4	500/1000	1000	1583.91	6000000	

	insured_zip	...	police_report_available	total_claim_amount	injury_claim	\
0	466132	...	YES	71610	6510	
1	468176	...	?	5070	780	
2	430632	...	NO	34650	7700	
3	608117	...	NO	63400	6340	
4	610706	...	NO	6500	1300	

	property_claim	vehicle_claim	auto_make	auto_model	auto_year	\
0	13020	52080	Saab	92x	2004	
1	780	3510	Mercedes	E400	2007	
2	3850	23100	Dodge	RAM	2007	
3	6340	50720	Chevrolet	Tahoe	2014	
4	650	4550	Accura	RSX	2009	

	fraud_reported	_c39
0	Y	NaN
1	Y	NaN
2	N	NaN
3	Y	NaN
4	N	NaN

[5 rows x 40 columns]

```
[4]: # Obtenemos características generales de la base de datos
print(f'La base de datos tiene {df.shape[0]} filas y {df.shape[1]} columnas')
print(f"{'Columnas':>30} | {'Tipo de Dato'}")
for col in df.columns:
```

```
print(f"{col:>30} | {df[col].dtype}")

df = df.drop(columns= "_c39")
```

La base de datos tiene 1000 filas y 40 columnas

Columnas	Tipo de Dato
months_as_customer	int64
age	int64
policy_number	int64
policy_bind_date	object
policy_state	object
policy_csl	object
policy_deductable	int64
policy_annual_premium	float64
umbrella_limit	int64
insured_zip	int64
insured_sex	object
insured_education_level	object
insured_occupation	object
insured_hobbies	object
insured_relationship	object
capital-gains	int64
capital-loss	int64
incident_date	object
incident_type	object
collision_type	object
incident_severity	object
authorities_contacted	object
incident_state	object
incident_city	object
incident_location	object
incident_hour_of_the_day	int64
number_of_vehicles_involved	int64
property_damage	object
bodily_injuries	int64
witnesses	int64
police_report_available	object
total_claim_amount	int64
injury_claim	int64
property_claim	int64
vehicle_claim	int64
auto_make	object
auto_model	object
auto_year	int64
fraud_reported	object
_c39	float64

```
[5]: # Esto ayuda a verificar que total_claim_amount se componga de las demás
      ↪columnas.
df['siniestralidad_total'] = df['injury_claim'] + df['property_claim'] +
      ↪df['vehicle_claim']
(df['siniestralidad_total'] == df['total_claim_amount']).sum()
```

```
[5]: np.int64(1000)
```

1. Clasificación y Explicación de las Columnas

Con el primer vistazo, identificamos que en total tenemos 1000 registros de los cuales se obtienen 40 variables, las cuales definiremos aquí para comprender su relevancia analítica.

1.1.1 A. Perfil del asegurado

- **months_as_customer (Meses como cliente) y age (Edad):** Permiten analizar la lealtad y el perfil demográfico. Ayudan a entender si los clientes más jóvenes o más nuevos presentan comportamientos de reclamo diferentes.
- **insured_sex, insured_education_level, insured_occupation, insured_relationship:** Variables demográficas estándar para segmentación. Permiten evaluar si el perfil socioeconómico influye en la siniestralidad.
- **insured_hobbies (Pasatiempos):** Variable conductual. Ciertos pasatiempos pueden estar correlacionados con perfiles de riesgo más altos o patrones inusuales en siniestros.
- **capital-gains (Ganancias de capital) y capital-loss (Pérdidas de capital):**

1.1.2 B. Detalles de la Póliza

- **policy_number e insured_zip:** Identificadores únicos
- **policy_bind_date (Fecha de emisión de póliza):** Crucial para calcular la antigüedad de la póliza al momento del accidente. (Reclamos muy cercanos a la fecha de contratación podrían ser alerta de fraude.)
- **policy_state (Estado de la póliza):** Distribución geográfica de la cartera.
- **policy_csl (Límites de responsabilidad combinados):** Define la cobertura máxima de la póliza por categoría (**Revisar si los montos vienen en millones/miles**).
- **policy_deductable (Deducible):** Monto que el cliente paga de su bolsillo.
- **policy_annual_premium (Prima anual):** Costo de la póliza.
- **umbrella_limit (Límite de póliza):** Cobertura de responsabilidad civil adicional.

1.1.3 C. Detalles del Incidente / Siniestro

- **incident_date (Fecha del incidente):** Permite evaluar factores de estacionalidad temporal (mes, día de la semana).
- **incident_type (Tipo de incidente) y collision_type (Tipo de colisión):** Clasifican la naturaleza del siniestro (**Tipo de colision tiene registros '?', verificar a qué se debe**).
- **incident_severity (Gravedad del incidente):** Los daños calificados como menores o mayores deben guardar relación directa con los costos del reclamo.
- **incident_hour_of_the_day (Hora del día):** La severidad se podría ver asociada con la hora del día en que ocurrió el siniestro.

- **incident_state, incident_city, incident_location:** Ubicación geográfica del evento para detectar zonas de alta siniestralidad o fraude concentrado.
- **authorities_contacted (Autoridades contactadas) y police_report_available (Reporte policial disponible):** La ausencia de intervención policial en incidentes severos representa una anomalía grave.
- **number_of_vehicles_involved (Vehículos involucrados) y witnesses (Testigos):** Ayudan a corroborar la consistencia física del relato del accidente.
- **property_damage (Daño a propiedad) y bodily_injuries (Lesiones corporales):** Indicadores de la magnitud de los daños colaterales.

1.1.4 D. Costos del Reclamo (Variables Financieras)

- **total_claim_amount (Monto total del reclamo):** La variable económica principal, se segrega en las siguientes variables:
 - **injury_claim** (Reclamo por lesiones).
 - * **property_claim** (Reclamo por daños a la propiedad).
 - **vehicle_claim** (Reclamo por daños al vehículo asegurado). ### E. Datos del Vehículo
 - **auto_make (Marca), auto_model (Modelo), auto_year (Año):** Determinan el valor comercial y el costo de refacciones.

1.1.5 F. Variable Objetivo y Columnas a Descartar

- **fraud_reported (Fraude reportado):** La variable objetivo. Todo el EDA girará en torno a cómo el resto de las variables interactúan con esta.
- **_c39:** Columna residual sin información útil generada comúnmente por errores de delimitación en la carga del archivo. **Debe ser eliminada.**

2 Análisis inicial (Comprender el comportamiento de la cartera)

1.1.6 Análisis perfil asegurados (AGREGAR)

```
[6]: #Gráficas para el perfil del asegurado:
variables = ['months_as_customer', 'age', 'insured_sex', 'capital-gains', 'capital-loss', 'insured_education_level',
            'insured_occupation', 'insured_relationship', 'insured_hobbies']
df_filtrado = df[variables]

fig, axes= plt.subplots(nrows = 3, ncols = 3,figsize=(20, 15))
axes = axes.flatten()

for i, var in enumerate(variables):
    # Trabajamos sobre el eje correspondiente
    ax = axes[i]

    # Si la variable es categórica, hacemos un grafico de barras
```

```

# Si hay varias categorias rotamos las etiquetas
if df_filtrado[var].dtype == "object":
    if df_filtrado[var].nunique() > 5:
        sns.countplot(
            x=var,
            data=df_filtrado,
            ax=ax,
            color = "purple"
        )

        ax.tick_params(axis='x', rotation=90)
        # Ajustamos títulos y etiquetas
        ax.set_title(f"Distribución de {var.capitalize()}")
        ax.set_xlabel("")
        ax.set_ylabel("Frecuencia")
    else:
        sns.countplot(
            x=var,
            data=df_filtrado,
            ax=ax,
            color = "purple"
        )
        # Ajustamos títulos y etiquetas
        ax.set_title(f"Distribución de {var.capitalize()}")
        ax.set_xlabel("")
        ax.set_ylabel("Frecuencia")
# Si la variable es numérica, hacemos un histograma
else:
    # Calcular los bins automáticamente para variables numéricas/discretas
    bins_choise = (
        len(df_filtrado[var].unique())
        if "año" in var or "fecha" in var or df_filtrado[var].nunique() < 10
        else 30
    )
    sns.histplot(
        data=df_filtrado,
        x=var,
        bins=bins_choise,
        ax=ax,
        color = "red"
    )
    # Ajustamos títulos y etiquetas
    ax.set_title(f"Histograma de {var.capitalize()}")
    ax.set_xlabel(var.capitalize())
    ax.set_ylabel("Frecuencia")

# Quitamos la leyenda individual de cada gráfico para que no estorbe

```

```

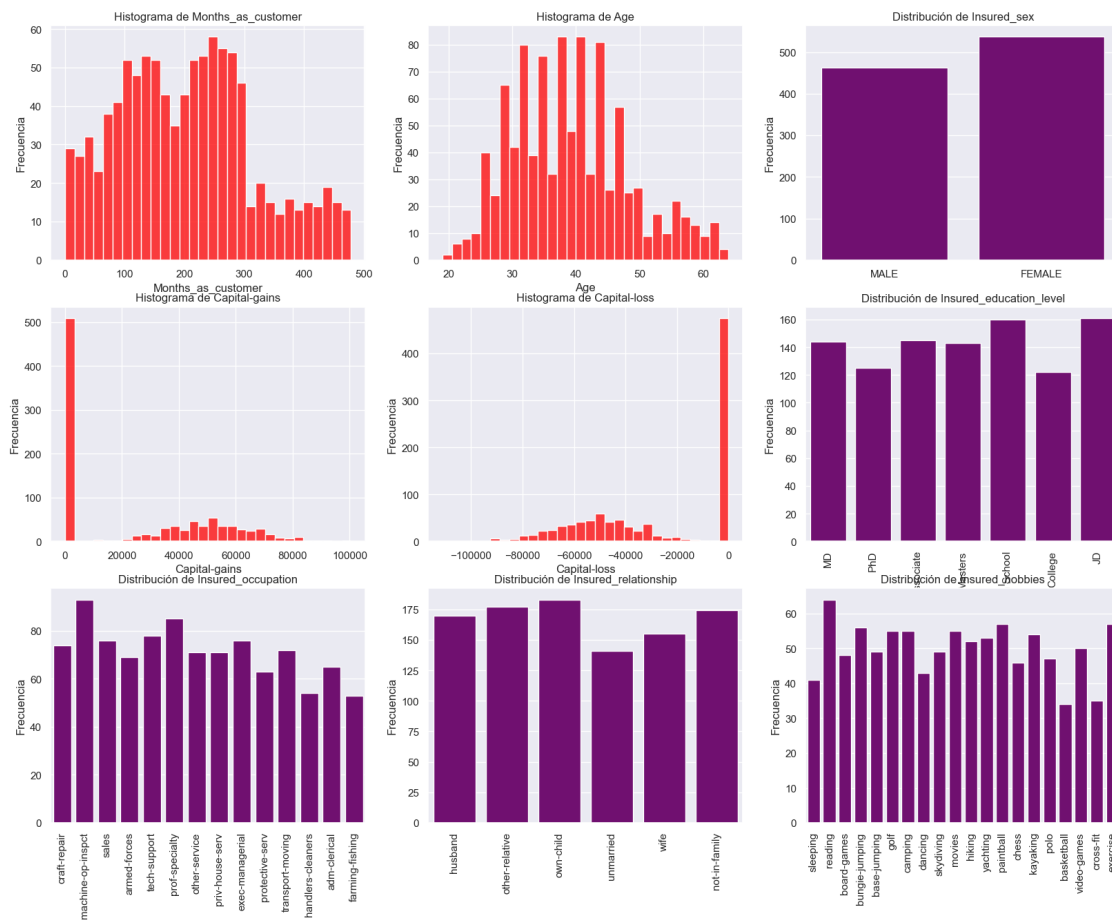
if ax.get_legend():
    ax.get_legend().remove()

fig.suptitle("Perfil del asegurado",
             fontsize=28, fontweight='bold', x=0.4, ha='left', y=1)

plt.show()

```

Perfil del asegurado



1.1.7 Análisis Detalles de la Póliza (AGREGAR)

```

[7]: #Gráficas para detalles de la póliza:
df["Año_Emission"] = pd.to_datetime(df["policy_bind_date"], format='%d/%m/%Y',
    ↪errors= "coerce").dt.year

```

```

variables = ["Año_Emision", "policy_state", "policy_csl", "policy_deductable",
            ↪ "policy_annual_premium", "umbrella_limit"]
df_filtrado = df[variables]

fig, axes= plt.subplots(nrows = 2, ncols = 3,figsize=(20, 15))
axes = axes.flatten()

for i, var in enumerate(variables):
    # Trabajamos sobre el eje correspondiente
    ax = axes[i]
    # Si la variable es categórica, hacemos un grafico de barras
    # Si hay varias categorias rotamos las etiquetas
    if df_filtrado[var].dtype == "object":
        if df_filtrado[var].nunique() > 5:
            sns.countplot(
                x=var,
                data=df_filtrado,
                ax=ax,
                color = "purple"
            )

            ax.tick_params(axis='x', rotation=90)
            # Ajustamos títulos y etiquetas
            ax.set_title(f"Distribución de {var.capitalize()}")
            ax.set_xlabel("")
            ax.set_ylabel("Frecuencia")
        else:
            sns.countplot(
                x=var,
                data=df_filtrado,
                ax=ax,
                color = "purple"
            )
            # Ajustamos títulos y etiquetas
            ax.set_title(f"Distribución de {var.capitalize()}")
            ax.set_xlabel("")
            ax.set_ylabel("Frecuencia")
    # Si la variable es numérica, hacemos un histograma
    else:
        # Calcular los bins automáticamente para variables numéricas/discretas
        bins_choise = (
            len(df_filtrado[var].unique())
            if "año" in var or "fecha" in var or df_filtrado[var].nunique() < 2
            else 30
        )
        sns.histplot(

```

```

        data=df_filtrado,
        x=var,
        bins=bins_choise,
        ax=ax,
        color = "red"
    )
    # Ajustamos títulos y etiquetas
    ax.set_title(f"Histograma de {var.capitalize()}")
    ax.set_xlabel(var.capitalize())
    ax.set_ylabel("Frecuencia")

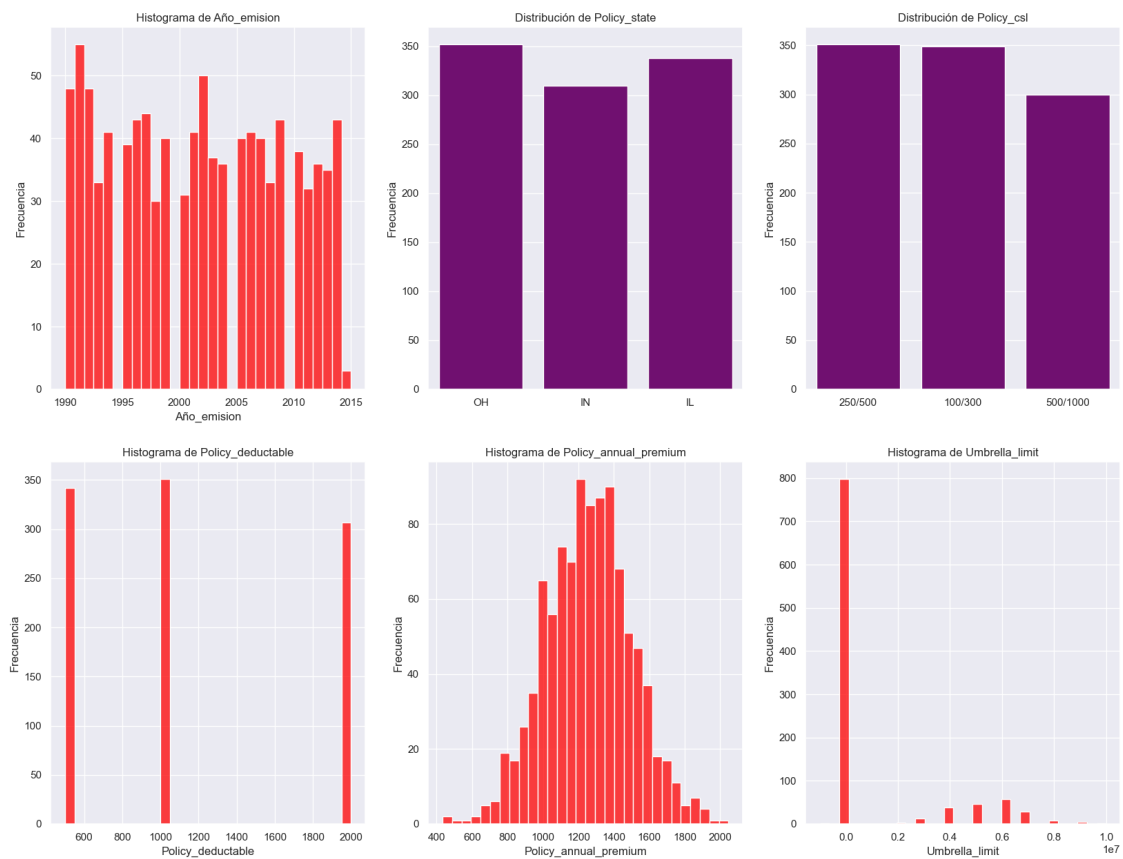
    # Quitamos la leyenda individual de cada gráfico para que no estorbe
    if ax.get_legend():
        ax.get_legend().remove()

fig.suptitle("Detalles de la Póliza",
            fontsize=28, fontweight='bold', x=0.4, ha='left', y=1)

plt.show()

```

Detalles de la Póliza



1.1.8 Análisis Detalles del Siniestro (AGREGAR)

```
[8]: #Gráficas para detalles del Siniestro:
variables = ["incident_type", "collision_type",
            ↪ "incident_severity", "incident_hour_of_the_day", "incident_state", "incident_city",
            ↪ "authorities_contacted", "police_report_available",
            ↪ "number_of_vehicles_involved", "witnesses", "property_damage",
            ↪ "bodily_injuries"]
df_filtrado = df[variables]

fig, axes= plt.subplots(nrows = 4, ncols = 3,figsize=(25, 25))
axes = axes.flatten()

for i, var in enumerate(variables):
    # Trabajamos sobre el eje correspondiente
    ax = axes[i]
    # Si la variable es categórica, hacemos un grafico de barras
    # Si hay varias categorias rotamos las etiquetas
    if df_filtrado[var].dtype == "object":
        if df_filtrado[var].nunique() > 5:
            sns.countplot(
                x=var,
                data=df_filtrado,
                ax=ax,
                color = "purple"
            )

            ax.tick_params(axis='x', rotation=90)
            # Ajustamos títulos y etiquetas
            ax.set_title(f"Distribución de {var.capitalize()}")
            ax.set_xlabel("")
            ax.set_ylabel("Frecuencia")
        else:
            sns.countplot(
                x=var,
                data=df_filtrado,
                ax=ax,
                color = "purple"
            )
            # Ajustamos títulos y etiquetas
            ax.set_title(f"Distribución de {var.capitalize()}")
            ax.set_xlabel("")
            ax.set_ylabel("Frecuencia")
```

```

# Si la variable es numérica, hacemos un histograma
else:
    # Calcular los bins automáticamente para variables numéricas/discretas
    bins_choise = (
        len(df_filtrado[var].unique())
        if "año" in var or "fecha" in var or df_filtrado[var].nunique() < 2
        else 30
    )
    sns.histplot(
        data=df_filtrado,
        x=var,
        bins=bins_choise,
        ax=ax,
        color = "red"
    )
    # Ajustamos títulos y etiquetas
    ax.set_title(f"Histograma de {var.capitalize()}")
    ax.set_xlabel(var.capitalize())
    ax.set_ylabel("Frecuencia")

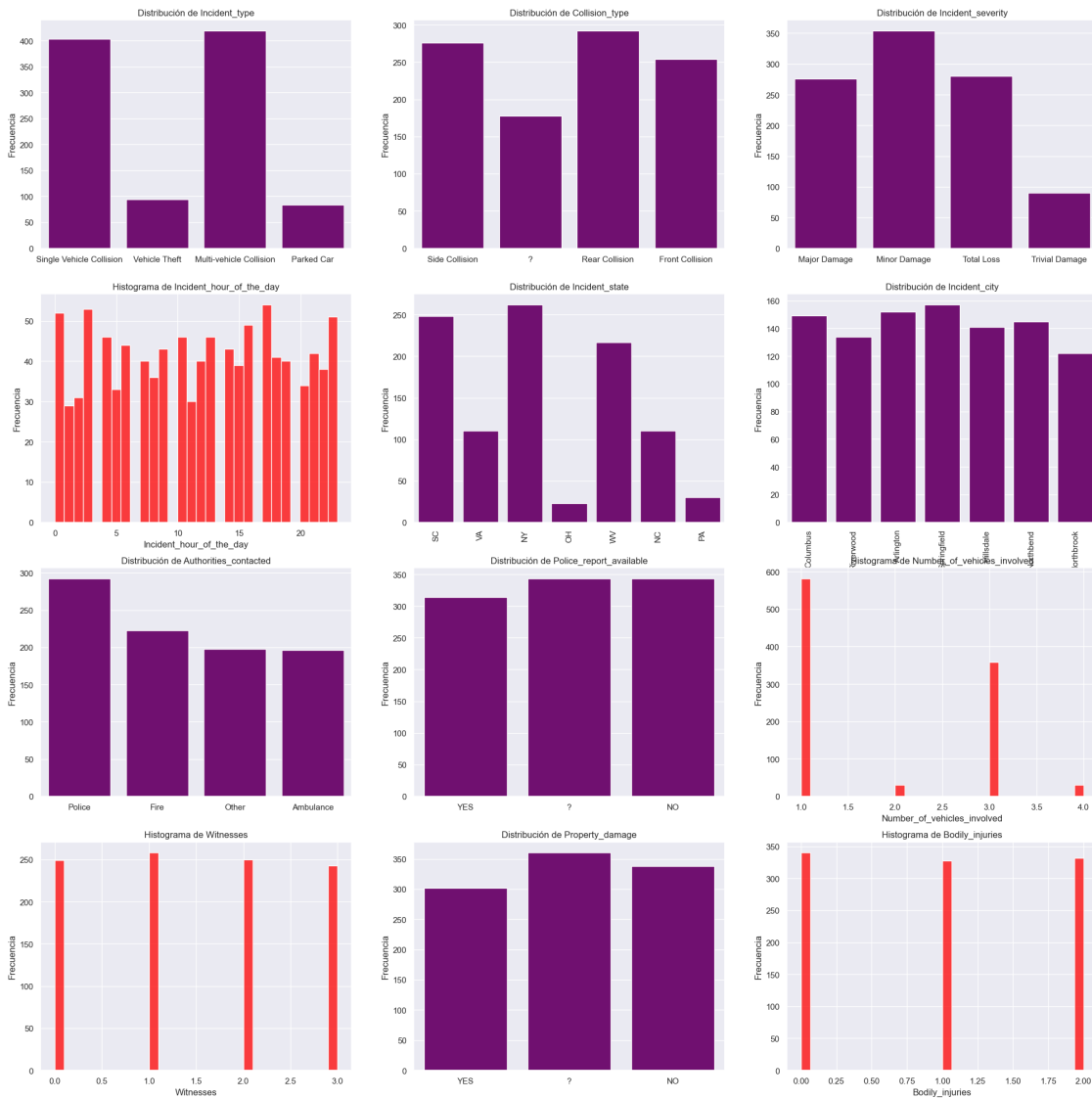
# Quitamos la leyenda individual de cada gráfico para que no estorbe
if ax.get_legend():
    ax.get_legend().remove()

fig.suptitle("Detalles del Siniestro",
            fontsize=28, fontweight='bold', x=0.4, ha='left', y=1)

plt.show()

```

Detalles del Siniestro



1.1.9 Análisis Costos del Reclamo (AGREGAR)

```
[9]: #Gráficas para Costos del Reclamo:
variables = ["total_claim_amount", "injury_claim", "property_claim", "vehicle_claim"]
df_filtrado = df[variables]
```

```

fig, axes= plt.subplots(nrows = 2, ncols = 2,figsize=(15, 15))
axes = axes.flatten()

for i, var in enumerate(variables):
    # Trabajamos sobre el eje correspondiente
    ax = axes[i]
    # Si la variable es categórica, hacemos un grafico de barras
    # Si hay varias categorias rotamos las etiquetas
    if df_filtrado[var].dtype == "object":
        if df_filtrado[var].nunique() > 5:
            sns.countplot(
                x=var,
                data=df_filtrado,
                ax=ax,
                color = "purple"
            )

            ax.tick_params(axis='x', rotation=90)
            # Ajustamos títulos y etiquetas
            ax.set_title(f"Distribución de {var.capitalize()}")
            ax.set_xlabel("")
            ax.set_ylabel("Frecuencia")
        else:
            sns.countplot(
                x=var,
                data=df_filtrado,
                ax=ax,
                color = "purple"
            )
            # Ajustamos títulos y etiquetas
            ax.set_title(f"Distribución de {var.capitalize()}")
            ax.set_xlabel("")
            ax.set_ylabel("Frecuencia")
    # Si la variable es numérica, hacemos un histograma
    else:
        # Calcular los bins automáticamente para variables numéricas/discretas
        bins_choise = (
            len(df_filtrado[var].unique())
            if "año" in var or "fecha" in var or df_filtrado[var].nunique() < 2
            else 30
        )
        sns.histplot(
            data=df_filtrado,
            x=var,
            bins=bins_choise,
            ax=ax,
            color = "red"

```

```

    )
    # Ajustamos títulos y etiquetas
    ax.set_title(f"Histograma de {var.capitalize()}")
    ax.set_xlabel(var.capitalize())
    ax.set_ylabel("Frecuencia")

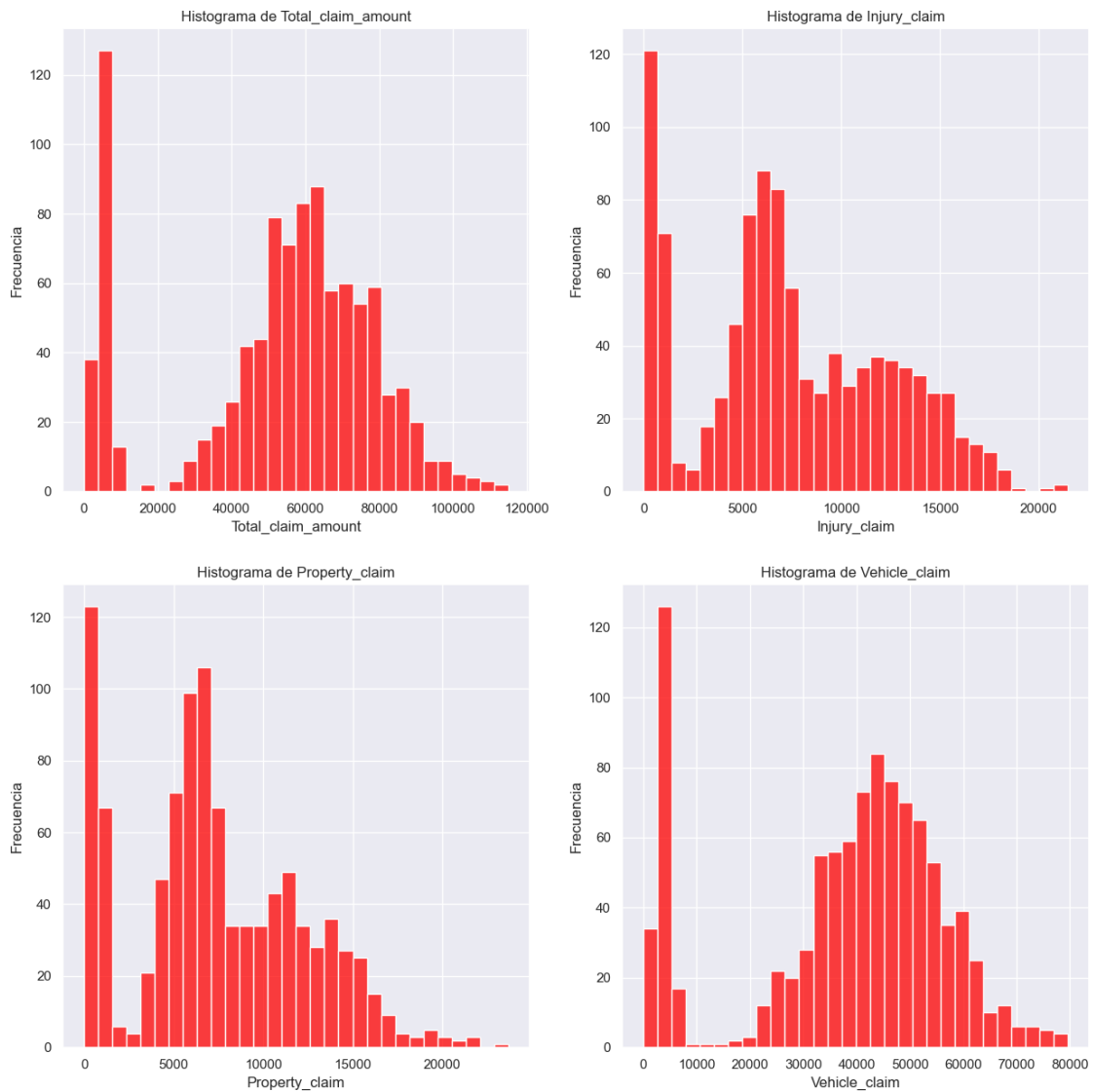
    # Quitamos la leyenda individual de cada gráfico para que no estorbe
    if ax.get_legend():
        ax.get_legend().remove()

fig.suptitle("Costos de Reclamo",
            fontsize=28, fontweight='bold', x=0.4, ha='left', y=1)

plt.show()

```

Costos de Reclamo



1.1.10 Análisis Datos del Vehículo (AGREGAR)

```
[10]: #Gráficas para Datos del Vehículo:
variables = ["auto_make", "auto_model", "auto_year"]
df_filtrado = df[variables]

fig, axes= plt.subplots(nrows = 1, ncols = 3, figsize=(25, 10))
```

```

axes = axes.flatten()

for i, var in enumerate(variables):
    # Trabajamos sobre el eje correspondiente
    ax = axes[i]
    # Si la variable es categórica, hacemos un grafico de barras
    # Si hay varias categorias rotamos las etiquetas
    if df_filtrado[var].dtype == "object":
        if df_filtrado[var].nunique() > 5:
            sns.countplot(
                x=var,
                data=df_filtrado,
                ax=ax,
                color = "purple"
            )

            ax.tick_params(axis='x', rotation=90)
            # Ajustamos títulos y etiquetas
            ax.set_title(f"Distribución de {var.capitalize()}")
            ax.set_xlabel("")
            ax.set_ylabel("Frecuencia")
        else:
            sns.countplot(
                x=var,
                data=df_filtrado,
                ax=ax,
                color = "purple"
            )
            # Ajustamos títulos y etiquetas
            ax.set_title(f"Distribución de {var.capitalize()}")
            ax.set_xlabel("")
            ax.set_ylabel("Frecuencia")
    # Si la variable es numérica, hacemos un histograma
    else:
        # Calcular los bins automáticamente para variables numéricas/discretas
        bins_choise = (
            len(df_filtrado[var].unique())
            if "año" in var or "fecha" in var or df_filtrado[var].nunique() < 2
            else 30
        )
        sns.histplot(
            data=df_filtrado,
            x=var,
            bins=bins_choise,
            ax=ax,
            color = "red"
        )

```

```

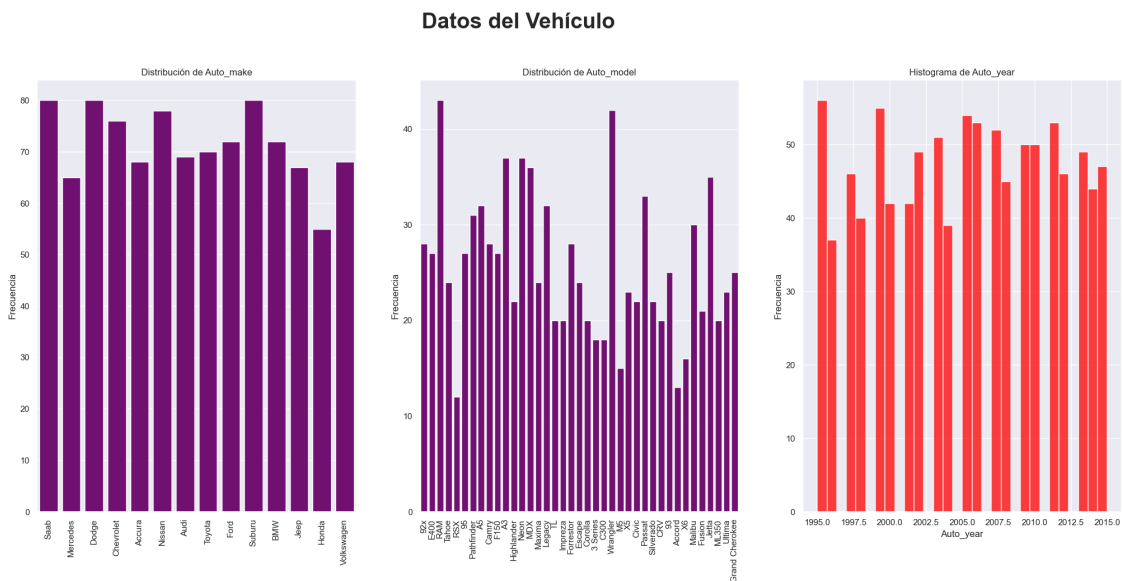
# Ajustamos títulos y etiquetas
ax.set_title(f"Histograma de {var.capitalize()}")
ax.set_xlabel(var.capitalize())
ax.set_ylabel("Frecuencia")

# Quitamos la leyenda individual de cada gráfico para que no estorbe
if ax.get_legend():
    ax.get_legend().remove()

fig.suptitle("Datos del Vehículo",
             fontsize=28, fontweight='bold', x=0.4, ha='left', y=1)

plt.show()

```



1.1.11 Análisis Variable Objetivo (AGREGAR)

```

[11]: # Gráfica para analizar variable objetivo (Podemos escoger el gráfico que más
      ↪ les agrade)
sns.countplot(data=df, x='fraud_reported', color = "purple")
plt.title('Distribución de fraude', fontweight='bold')
plt.xlabel('fraud_reported')
plt.ylabel('Frecuencia absoluta')
plt.show()

conteo = df['fraud_reported'].value_counts()

```

```

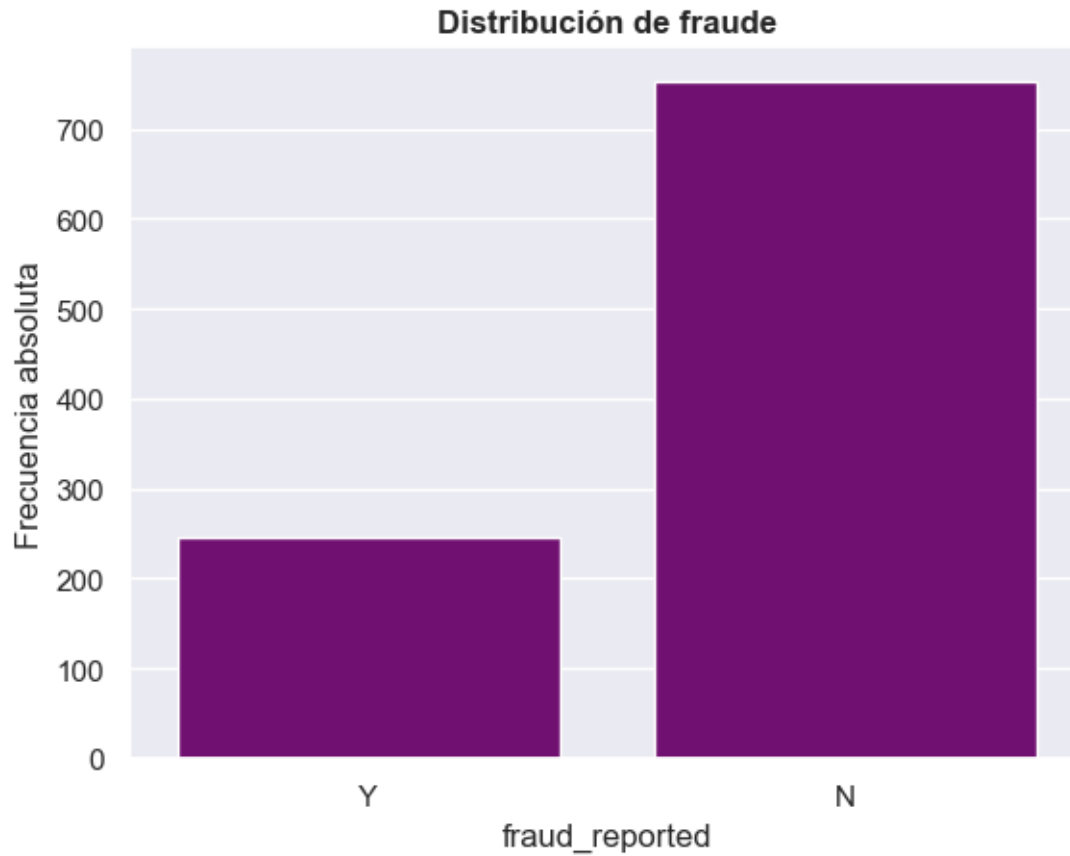
fig = go.Figure()

fig.add_trace(
    go.Bar(
        x=conteo.index,
        y=conteo.values,
        marker_color=px.colors.qualitative.Plotly[:len(conteo)],
        opacity=0.92,
        text=conteo.values,
        textposition='outside',
        texttemplate='%{text:,.0f}',
        cliponaxis=False,
        hovertemplate='<b>%(x)</b><br>Pólizas: %(y:,.0f)<extra></extra>'
    )
)

fig.update_layout(
    title='Insurance: distribución fraud_reported',
    xaxis_title='fraud_reported',
    yaxis_title='Frecuencia absoluta',
    width=1000,
    height=600
)

fig.show()

```



3 Relación con el Fraude

1.1.12 Análisis inicial (AGREGAR)

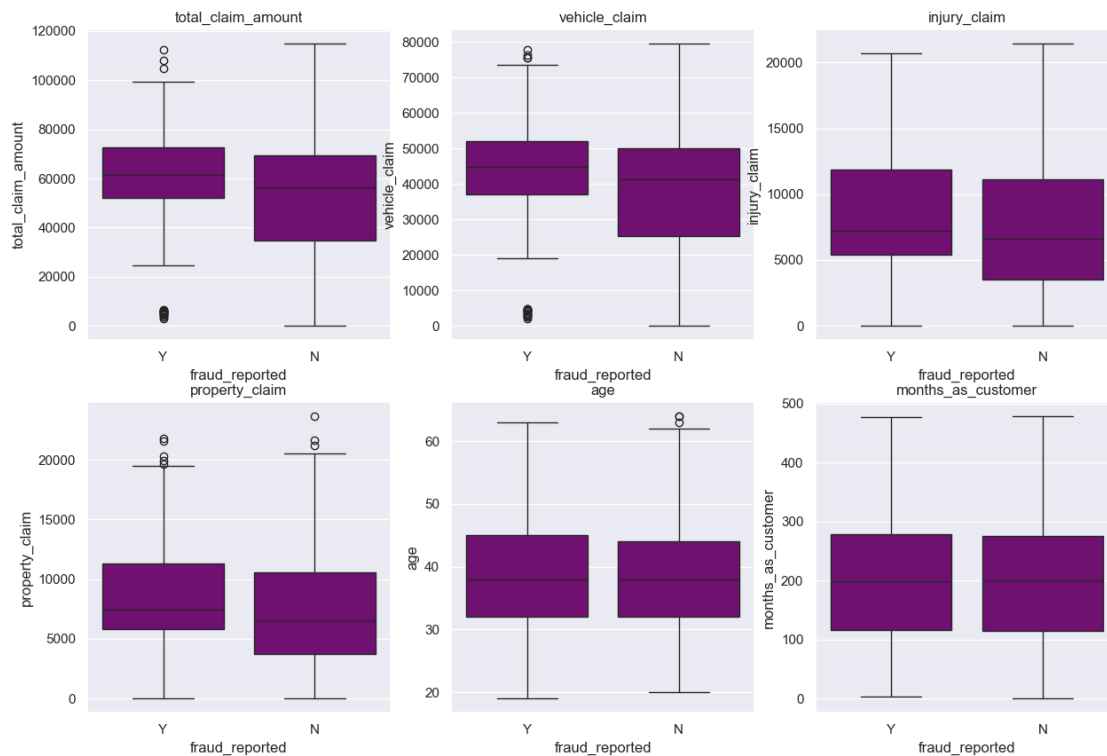
```
[ ]: # Análisis de variables de acuerdo a categorización de fraude
variables = [
    'total_claim_amount',
    'vehicle_claim',
    'injury_claim',
    'property_claim',
    'age',
    'months_as_customer'
]

fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(15,10))
for var, axes in zip(variables, axes.flat):
    sns.boxplot(data=df, x='fraud_reported', y=var, ax=axes, color = 'purple')
    axes.set_title(var)
```

```
fig.suptitle("Variables vs Fraude",
             fontsize=28, fontweight='bold', x=0.37, ha='left', y=1)

plt.show()
```

Variables vs Fraude



1.1.13 Agregar comentarios

```
[ ]: # Siniestralidad de acuerdo a etiqueta de Fraude
fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(21, 10))

sns.violinplot(
    data=df,
    x="fraud_reported",
    y="total_claim_amount",
    hue="fraud_reported",
    palette="Set2",
    inner="box",          # agrega boxplot interno
    legend=False,
    ax = axes[0])
```

```

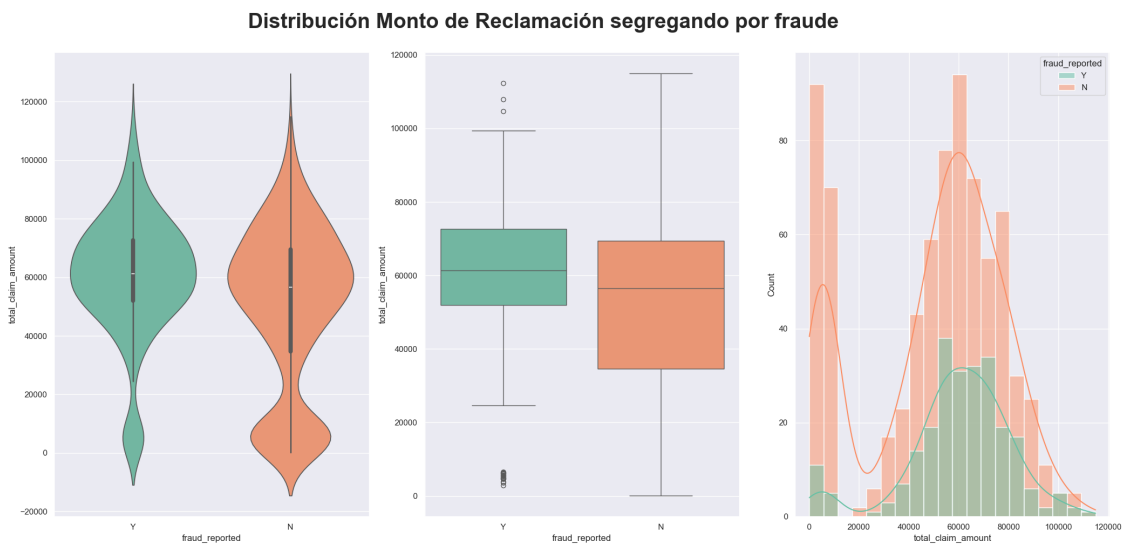
)

sns.boxplot(data=df,
             x="fraud_reported",
             hue="fraud_reported",
             y="total_claim_amount",
             palette="Set2",
             ax = axes[1])

sns.histplot(data=df,
             x = "total_claim_amount",
             hue = "fraud_reported",
             palette = "Set2",
             ax = axes[2],
             kde = True)

fig.suptitle("Distribución Monto de Reclamación segregando por fraude",
             fontsize=28, fontweight='bold', x=0.22, ha='left', y=1)
plt.tight_layout()
plt.show()

```



1.1.14 Agregar comentario

```
[ ]: # Función para crear subplot con px
def agregar_grafico_a_subplot(
    df, columna_caracteristica, fig_subplots, fila, columna,
    ↪columna_fraude="fraud_reported"
):
    """
    Función que genera gráficos exprés temporales y donde se define un subplot,
    ↪global donde se van metiendo todos los temporales\n

    Parámetros:
    -----
    - df: El DataFrame que contiene los datos
    - columna_caracteristica: (str) Nombre de la columna a analizar
    - fig_subplots: (variable tipo gráfico [make_subplots]) Figura global donde
    ↪se insertarán todos los gráficos que se crearon temporalmente
    - fila: (int) Índice de la fila de la cuadrícula donde se insertará el
    ↪gráfico
    - columna: (int) Índice de la columna de la cuadrícula donde se insertará
    ↪el gráfico
    - columna_fraude: (str) Nombre de la columna objetivo
    """

    # Tabla Contingente
    tabla = pd.crosstab(
        df[columna_caracteristica], df[columna_fraude], normalize="index"
    ).reset_index()
    # Tabla a formato largo
    tabla_long = tabla.melt(
        id_vars=columna_caracteristica,
        var_name=columna_fraude,
        value_name="proporcion",
    )

    # Creamos un grafico temporal con Plotly Express
    fig_temporal = px.bar(
        tabla_long,
        x=columna_caracteristica,
        y="proporcion",
        color=columna_fraude,
        barmode="stack",
        text_auto=".1%",
    )

    # Mover los trazos del gráfico temporal al subplot global
    for trace in fig_temporal.data:
```

```

        # Si no es la primera celda (1,1), desactivamos la leyenda para evitar
        ↪ que se duplique
        if fila > 1 or columna > 1:
            trace.showlegend = False
        # Por como funciona px, agregamos el grafico temporal a uno que tenga
        ↪ todos
        fig_subplots.add_trace(trace, row=fila, col=columna)

        # 4. Actualizar los títulos de los ejes para esta cuadrícula específica
        fig_subplots.update_xaxes(title_text=columna_caracteristica, row=fila,
        ↪ col=columna)

```

[79]: # Creamos gráficos de proporciones por Fraude

```

variables_a_analizar = [
    "insured_education_level",
    "insured_hobbies",
    "insured_sex",
    "incident_severity",
    "incident_city",
    "collision_type",
    "incident_hour_of_the_day",
    'insured_occupation',
]

# Definimos la estructura en la que se presentará el gráfico
filas = 2
columnas = 4

fig_global = make_subplots(
    rows=filas,
    cols=columnas,
    subplot_titles=[f"Fraude por {var}" for var in variables_a_analizar],
    shared_yaxes=True, # Compartir el eje Y para comparar proporciones
    ↪ fácilmente
)

# 3. (usamos enumerate para saber en qué fila/columna meter cada gráfica)
for indice, caracteristica in enumerate(variables_a_analizar):
    # Calcular la posición en la matriz (1-indexed para Plotly)
    fila = (indice // columnas) + 1
    columna = (indice % columnas) + 1

    # Ponemos el gráfico de la característica actual en el lienzo global
    agregar_grafico_a_subplot(
        df=df,

```

```

        columna_caracteristica=caracteristica,
        fig_subplots=fig_global,
        fila=fila,
        columna=columna,
    )

# Ajustes finales de la gráfica global
fig_global.update_layout(
    title_text="Análisis de Proporciones de Fraude",
    height=1000, # Ajusta el alto total para que no se amontonen
    width=1800, # Ajusta el ancho total
    barmode="stack", # Asegura que todos los subplots mantengan el estilo
    ↪apilado
    yaxis_tickformat=".0%",
)

fig_global.show()

```

```

[ ]: # Tasa de fraude por hora del incidente (arriba ya se puso, cuál prefieren?)
df['fraud_ind'] = df['fraud_reported'].map({'Y': 1, 'N': 0})
tabla_hora = (
    df.groupby("incident_hour_of_the_day")
      .agg(
          n=("fraud_ind", "size"),
          fraudes=("fraud_ind", "sum"),
          tasa_fraude=("fraud_ind", "mean")
      )
      .reset_index()
)

tabla_hora["tasa_fraude_pct"] = 100 * tabla_hora["tasa_fraude"]

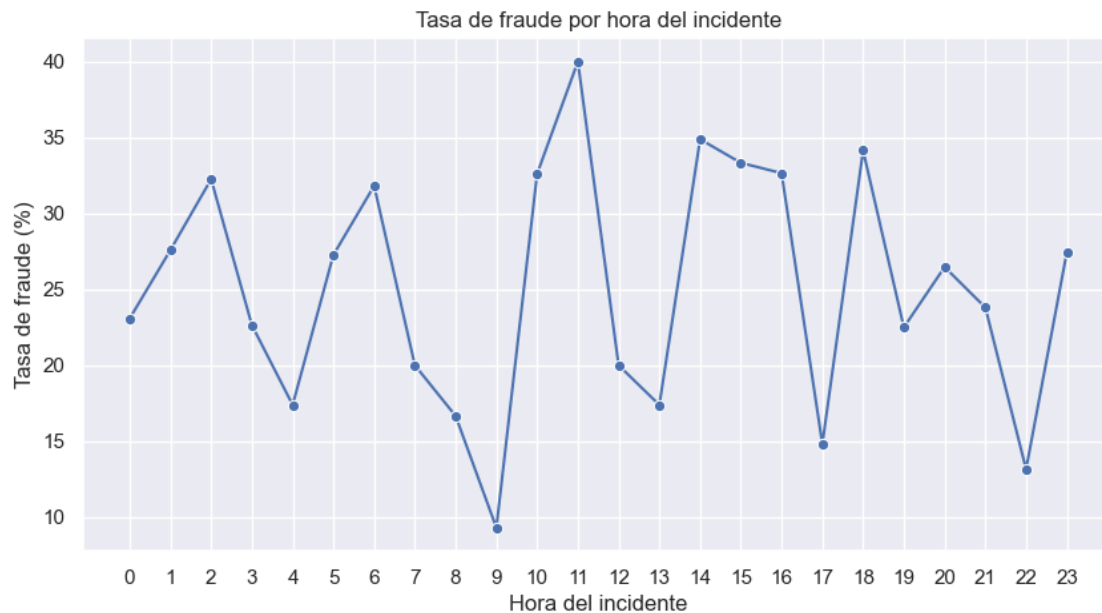
display(tabla_hora)

plt.figure(figsize=(10, 5))
sns.lineplot(
    data=tabla_hora,
    x="incident_hour_of_the_day",
    y="tasa_fraude_pct",
    marker="o"
)

plt.title("Tasa de fraude por hora del incidente")
plt.xlabel("Hora del incidente")
plt.ylabel("Tasa de fraude (%)")
plt.xticks(range(0, 24))
plt.show()

```

	incident_hour_of_the_day	n	fraudes	tasa_fraude	tasa_fraude_pct
0	0	52	12	0.230769	23.076923
1	1	29	8	0.275862	27.586207
2	2	31	10	0.322581	32.258065
3	3	53	12	0.226415	22.641509
4	4	46	8	0.173913	17.391304
5	5	33	9	0.272727	27.272727
6	6	44	14	0.318182	31.818182
7	7	40	8	0.200000	20.000000
8	8	36	6	0.166667	16.666667
9	9	43	4	0.093023	9.302326
10	10	46	15	0.326087	32.608696
11	11	30	12	0.400000	40.000000
12	12	40	8	0.200000	20.000000
13	13	46	8	0.173913	17.391304
14	14	43	15	0.348837	34.883721
15	15	39	13	0.333333	33.333333
16	16	49	16	0.326531	32.653061
17	17	54	8	0.148148	14.814815
18	18	41	14	0.341463	34.146341
19	19	40	9	0.225000	22.500000
20	20	34	9	0.264706	26.470588
21	21	42	10	0.238095	23.809524
22	22	38	5	0.131579	13.157895
23	23	51	14	0.274510	27.450980



```

[ ]: # (arriba ya se puso, cuál prefieren?)
df['fraude_bin'] = (df['fraud_reported'] == 'Y').astype(int)

# Tasa de fraude por ocupación
heat = (
    df.groupby('insured_occupation')['fraude_bin']
      .mean()
      .sort_values(ascending=False)
      .to_frame()
)

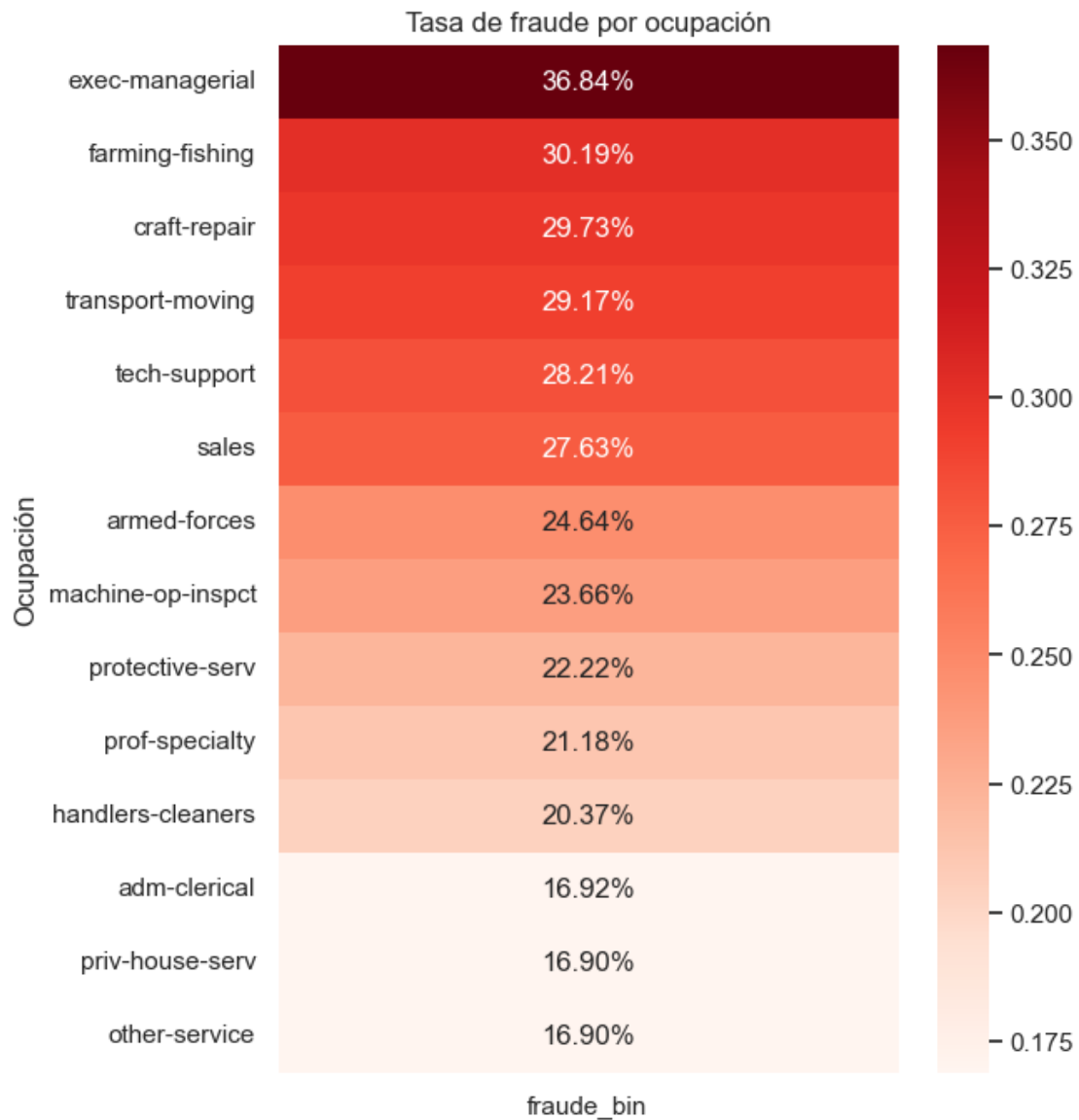
plt.figure(figsize=(6,8))

sns.heatmap(
    heat,
    annot=True,
    fmt='.2%',
    cmap='Reds'
)

plt.title('Tasa de fraude por ocupación')
plt.xlabel('')
plt.ylabel('Ocupación')

plt.show()

```



```
[86]: #insured_occupation
#Comparación de poblaciones a través de densidades: charges por sexo

fig, axes = plt.subplots(1, 2, figsize=(18, 5))

sns.kdeplot(data=df, x="total_claim_amount", hue="insured_occupation",
            fill=True, ax = axes[0])
axes[0].set_title('Monto total de reclamación: Comparación por profesion',
            fontsize=12, fontweight='bold')
axes[0].set_xlabel('Severidad del Incidente')
```

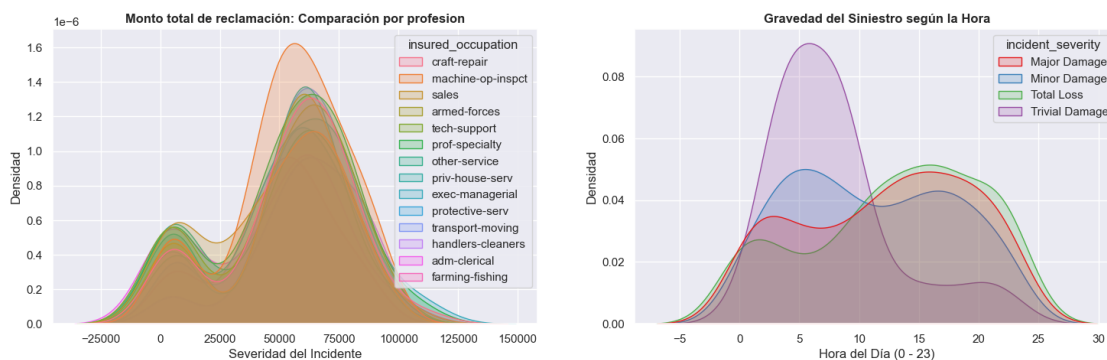
```

axes[0].set_ylabel('Densidad')

sns.kdeplot(data=df, x='incident_hour_of_the_day', hue='incident_severity',
            common_norm=False, fill=True, palette='Set1', alpha=0.2, ax=axes[1])
axes[1].set_title('Gravedad del Siniestro según la Hora', fontsize=12,
                fontweight='bold')
axes[1].set_xlabel('Hora del Día (0 - 23)')
axes[1].set_ylabel('Densidad')

plt.show()

```



Pasos que faltan para profundizar:

- **Completar análisis actual:** Revisar la asimetría, curtosis y sesgo de las distribuciones que hemos graficado.
- **Análisis de NA's:** En general no hay NA's en el dataframe, sin embargo, hay 3 variables que detallan el siniestro que tienen una cantidad significativa de registros '?', se podría analizar si tiene sentido esto o se debe a algún error que debamos gestionar.
- **Análisis de Siniestralidad por grupos con mayor fraude:** Esto permite analizar el impacto que tienen estos eventos.
- **Análisis de Correlación entre variables:** Esto debe de ser coherente con todo lo que hemos analizado con anterioridad (se debe hacer encoding con las variables categóricas)
- **Pruebas de hipótesis:** Se deben realizar pruebas de hipótesis sobre los comportamientos que observamos en los gráficos
 - **Prueba de Chi-Square Statistic:** Existe asociación estadísticamente significativa entre las variables (Esta prueba de hipótesis se puede usar en todo el apartado donde hacemos gráficos de porcentaje de fraude)
 - **Prueba no paramétrica U de Mann-witney o Prueba de homogeneidad de varianza (levene y bartlett):** Comparación de poblaciones (Si hacemos el análisis de

Siniestralidad) podemos realizar prueba para ver si la siniestralidad difiere por grupos fraudulentos o no.